

VGG16_Quantization_Aware_Training_2bit4bit

December 12, 2025

```
[ ]: import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn

import torchvision
import torchvision.transforms as transforms

from models import *

global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')

batch_size = 128
model_name = "VGG16_project"
model = VGG16_project()

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243,
↪0.262])

train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
```

```

download=True,
transform=transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    normalize,
]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,
↳shuffle=True, num_workers=2)

test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size,
↳shuffle=False, num_workers=2)

print_freq = 100 # every 100 batches, accuracy printed. Here, each batch
↳includes "batch_size" data points
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)

```

```

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()

    if i % print_freq == 0:
        print('Epoch: [{0}] [{1}/{2}]\t'
              'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
              'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
              'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
              'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                  epoch, i, len(trainloader), batch_time=batch_time,
                  data_time=data_time, loss=losses, top1=top1))

def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))

```

```

        top1.update(prec.item(), input.size(0))

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()

        if i % print_freq == 0: # This line shows how frequently print out
            ↪ the status. e.g., i%5 => every 5 batch, prints out
            print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                      i, len(val_loader), batch_time=batch_time, loss=losses,
                      top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg

def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res

class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):

```

```

        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count

def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)

```

```

[2]: def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120_
    ↪epochs"""
    adjust_list = [100, 120]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1

```

```

[ ]: # This cell won't be given, but students will complete the training

lr = 4e-2
weight_decay = 1e-4
epochs = 150
best_prec = 0

#model = nn.DataParallel(model).cuda()
model.cuda()
criterion = nn.CrossEntropyLoss().cuda()
optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9, ↪
    ↪weight_decay=weight_decay)
#cudnn.benchmark = True

if not os.path.exists('result'):
    os.makedirs('result')
fdir = 'result/' + str(model_name)
if not os.path.exists(fdir):
    os.makedirs(fdir)

```

```

for epoch in range(0, epochs):
    adjust_learning_rate(optimizer, epoch)

    train(trainloader, model, criterion, optimizer, epoch)

    # evaluate on test set
    print("Validation starts")
    prec = validate(testloader, model, criterion)

    # remember best precision and save checkpoint
    is_best = prec > best_prec
    best_prec = max(prec, best_prec)
    print('best acc: {:.1f}'.format(best_prec))
    save_checkpoint({
        'epoch': epoch + 1,
        'state_dict': model.state_dict(),
        'best_prec': best_prec,
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)

```

```

[2]: PATH = "./result/VGG16_quant2b4b/model_best.pth.tar"
model_name = "VGG16_project_2b4b"
model = VGG16_project()
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%) \n'.format(
    correct, len(testloader.dataset),
    100. * correct / len(testloader.dataset)))

```

Test set: Accuracy: 8939/10000 (89%)

```
[ ]: class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

##### Save inputs from selected layer #####
save_output = SaveOutput()
i = 0
counter = 0
for layer in model.modules():
    i+=1
    # print(i, "-th layer:", layer)
    if isinstance(layer, QuantConv2d):
        print(i, "-th layer prehooked")
        counter+=1
        print(layer, counter)
        layer.register_forward_pre_hook(save_output)
#####

dataiter = iter(testloader)
images, labels = next(dataiter)
images = images.to(device)
out = model(images)
```

```
[4]: w_bit = 4
weight_q = model.features[27].weight_q # quantized value is stored during the
    ↪ training
w_alpha = model.features[27].weight_quant.wgt_alpha # alpha is defined in
    ↪ your model already. bring it out here
w_delta = w_alpha/(2**(w_bit-1)-1) # delta can be calculated by using alpha
    ↪ and w_bit
weight_int = weight_q/w_delta # w_int can be calculated by weight_q and w_delta
# print(weight_int) # you should see clean integer numbers
print(f"Weight Int Shape: {weight_int.shape}")
```

Weight Int Shape: torch.Size([16, 16, 3, 3])

```
[ ]: print(weight_int[0][:][:][:])
```

```
[6]: x_bit = 2
x = save_output.outputs[8][0] # input of the 2nd conv layer
print(f"Input Shape: {x.shape}")
x_alpha = model.features[27].act_alpha
```

```

x_delta = x_alpha/(2**x_bit-1)

act_quant_fn = act_quantization(x_bit) # define the quantization function
x_q = act_quant_fn(x, x_alpha) # create the quantized value for x

x_int = x_q/x_delta
print(f"Input Int Sample: {x_int[0,0,0,:5]}")

```

```

Input Shape: torch.Size([128, 16, 4, 4])
Input Int Sample: tensor([2.0000, 3.0000, 2.0000, 2.0000], device='cuda:0',
      grad_fn=<SelectBackward0>)

```

[]:

[7]: *#### input floating number / weight quantized version*

```

conv_ref = torch.nn.Conv2d(in_channels = 16, out_channels=16, kernel_size = 3,
    ↪padding=1, bias = False)
conv_ref.weight = torch.nn.parameter.Parameter(weight_int)
output_int = conv_ref(x_int)

output_recovered = output_int*x_delta*w_delta
relu = nn.ReLU(inplace=True)
relu_output_recovered = relu(output_recovered)

output_ref = save_output.outputs[9][0]

```

[8]: difference = abs(output_ref - relu_output_recovered)
print(difference.mean()) *## It should be small, e.g., 2.3 in my trained model*

```

tensor(1.1046e-06, device='cuda:0', grad_fn=<MeanBackward0>)

```

[17]: x_pad = torch.zeros(128, 16, 6, 6).to(x_int.device)
x_pad[:, :, 1:5, 1:5] = x_int
x_pad_in = x_pad[0]
X = x_pad_in.reshape(x_pad_in.size(0), -1)
X.size()

[17]: torch.Size([16, 36])

[15]: x_pad.shape

[15]: torch.Size([128, 16, 6, 6])

[18]: bit_precision = 2
file_path = 'activation2b4b.txt'

with open(file_path, 'w') as f:


```

f.write('//Format:[Row15_MSB...Row0_LSB](Each-row-is-4-bit)\n')
f.write('//Time-step-0-to-35\n')

for i in range(X.size(1)):
    line_bin = ""

    for j in range(X.size(0)):
        val = int(round(X[15-j, i].item()))

        val_2s = val & 0xF

        line_bin += f'{val_2s:04b}'

    f.write(line_bin + '\n')

```

```
[19]: weight_int.size()
```

```
[19]: torch.Size([16, 16, 3, 3])
```

```
[20]: W = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))
W.size()
```

```
[20]: torch.Size([16, 16, 9])
```

```
[21]: W = weight_int.reshape(weight_int.size(0), weight_int.size(1), -1)
W.size()
```

```
[21]: torch.Size([16, 16, 9])
```

```

[22]: bit_precision = 4
file_path = 'weight2b4b.txt'

och_tile=8
with open(file_path, 'w') as f:
    f.write('//Format:[Row15_MSB...Row0_LSB]for_specific_Col_and_Kernel_idx\n')
    f.write('//Layout:K_ij->Output_Ch(Col)->Input_Ch(Row)\n')

    for kij in range(W.size(2)):
        for och_t in range(int(W.size(0)/och_tile)):
            for i in range(och_tile):
                line_bin = ""

                for j in range(W.size(1)):
                    val = int(round(W[i+och_t*och_tile, 15-j, kij].item()))

                    val_2s = val & 0xF

```

```

        line_bin += f'{val_2s:04b}'

    f.write(line_bin + '\n')

```

```
[23]: W[0,:,0]
```

```
[23]: tensor([-6.,  5., -2., -0., -2., -3., -4., -2.,  4.,  1., -4., -4., -2.,  1.,
          -3.,  3.], device='cuda:0', grad_fn=<SelectBackward0>)
```

```
[24]: psum = torch.zeros(16, X.size(1), 9).to(device)
      psum.shape
```

```
[24]: torch.Size([16, 36, 9])
```

```
[25]: for kij in range(9):
      W_k = W[:, :, kij]

      psum_slice = torch.matmul(W_k.float(), X.float())

      psum[:, :, kij] = psum_slice

bit_precision = 16
file_path = 'psum.txt'

with open(file_path, 'w') as f:
    f.write('//Format:[Col7_MSB...Col0_LSB] (Each 16-bit)\n')
    f.write('//Order:LoopKernel(9)->LoopTime(36)\n')

    for kij in range(9):
        for och_t in range(int(W.size(0)/och_tile)):
            for i in range(psum.size(1)):
                line_bin = ""

                for j in range(och_tile):
                    val = int(round(psum[7-j+och_t*och_tile, i, kij].item()))

                    val_2s = val & 0xFFFF

                    line_bin += f'{val_2s:016b}'

                f.write(line_bin + '\n')

```

```
[26]: out = output_int[0]
      out = out.reshape(out.size(0), -1)
```

```
out.shape
```

```
[26]: torch.Size([16, 16])
```

```
[41]: out = F.relu(out)
```

```
[42]: # import torch
# import torch.nn.functional as F

# # out_relu = F.relu(output_int)
# out = output_int[0]
# out = out.reshape(out.size(0), -1)

bit_precision = 16
file_path = 'output2b4b.txt'

with open(file_path, 'w') as f:
    f.write('//Format:[Col7_MSB...Col0_LSB] (Each16-bit)\n')
    f.write('//FinalOutput\n')

    for och_t in range(int(W.size(0)/och_tile)):

        for i in range(out.size(1)):
            line_bin = ""

            # Loop Channel (Row 7 -> Row 0)
            for j in range(och_tile):
                val = int(round(out[7-j+och_t*och_tile, i].item()))

                val_bin = val & 0xFFFF

                line_bin += f'{val_bin:016b}'

            f.write(line_bin + '\n')
```

```
[28]: file_path = 'output2b4b.txt'
output_hex_path = 'output_hex.txt'

with open(file_path, 'r') as f_in, open(output_hex_path, 'w') as f_out:
    for line in f_in:
        line = line.strip()
        if line.startswith('//'):
            # f_out.write(line + '\n')
            continue
        else:
            # Convert 128-bit binary string to hex
```

```
hex_value = hex(int(line, 2))[2:].upper().zfill(32)
f_out.write(hex_value + '\n')
```

```
[ ]: #
w = weight_int[0]          # : [in_channels, 3, 3]
x = x_pad[0,:, 0:3, 0:3]   # : [in_channels, 3, 3]

#
w_flat = w.reshape(w.size(0), -1)    # [in_channels, 9]
x_flat = x.reshape(x.size(0), -1)    # [in_channels, 9]

print(w_flat.shape)
print(x_flat.shape)

# output_int:
oout = torch.sum(w_flat * x_flat, dim=0) # [9]

print(oout)
```

```
[ ]: import torch

def simulate_systolic_array_detailed(x_pad_in, weight_int, och_tile_size=8):
    """
    Simulate systolic array with detailed cycle-by-cycle output printing
    Ensuring all operations are on the same device
    """
    # Ensure both tensors are on the same device
    device = x_pad_in.device
    weight_int = weight_int.to(device)

    print("=== Systolic Array Simulation with Detailed Output ===")
    print(f"Device: {device}")
    print(f"Input shape: {x_pad_in.shape}")
    print(f"Weight shape: {weight_int.shape}")
    print(f"OCH tile size: {och_tile_size}")

    # Prepare input data - extract 4x4 output region from 6x6 padded input
    input_patches = torch.zeros(16, 4, 4, 3, 3, device=device) # [in_ch,
↪ H_out, W_out, kH, kW]

    for i in range(4):
        for j in range(4):
            input_patches[:, i, j, :, :] = x_pad_in[:, i:i+3, j:j+3]

    # Reshape to [in_ch, spatial_pos, kernel_elements]
    input_matrix = input_patches.view(16, 16, 9) # [16, 16, 9]
```

```

# Reshape weights to [out_ch, in_ch, kernel_elements]
W = weight_int.view(weight_int.size(0), weight_int.size(1), 9) # [16, 16, 9]
↪9]

total_output_channels = W.size(0) # 16
num_och_tiles = total_output_channels // och_tile_size # 16//8 = 2

# Storage for partial sums - ensure on same device
psum_memory = torch.zeros(total_output_channels, 16, 9, device=device) # 16
↪[out_ch, spatial_pos, kij]

print(f"\nSimulation parameters:")
print(f"- Total output channels: {total_output_channels}")
print(f"- Number of OCH tiles: {num_och_tiles}")
print(f"- Spatial positions (nij): {input_matrix.size(1)}")
print(f"- Kernel elements (kij): {input_matrix.size(2)}")

cycle_count = 0

# Exact loop structure matching your Verilog testbench
for kij in range(9): # Kernel element index (0-8)
    print(f"\n--- Processing kij = {kij} ---")

    for och_t in range(num_och_tiles): # Output channel tile (0 to 1)
        ↪num_och_tiles-1)
        print(f" --- Processing och_tile = {och_t} ---")

        # Calculate output channel range for this tile
        och_start = och_t * och_tile_size
        och_end = (och_t + 1) * och_tile_size

        # Extract weights for current tile and kernel element
        w_tile_kij = W[och_start:och_end, :, kij] # [8, 16]
        print(f" Weight tile shape for kij={kij}: {w_tile_kij.shape}")

        # Process each time step (spatial position)
        for nij in range(16): # Time steps/spatial positions (0-15)
            # Get input at this position and kernel element
            x_nij_kij = input_matrix[:, nij, kij] # [16]

            # Compute matrix multiplication: [och_tile_size] × [in_ch]
            ↪[och_tile_size, in_ch] × [in_ch]
            psum_result = torch.matmul(w_tile_kij, x_nij_kij) # [8]

            # Store partial sums in memory
            psum_memory[och_start:och_end, nij, kij] = psum_result

```

```

        # Print first few cycles in detail
        if cycle_count < 20:
            print(f"    Cycle {cycle_count}: nij={nij}")
            # Convert to 16-bit signed integers and then to hex
            for idx, val in enumerate(psum_result):
                och_index = och_start + idx
                # Convert to 16-bit signed integer (same as Verilog)
                int_val = int(round(val.item()))
                # Handle negative numbers for 16-bit two's complement
                if int_val < 0:
                    hex_val = format(int_val & 0xFFFF, '04X') # 16-bit
→two's complement

                else:
                    hex_val = format(int_val & 0xFFFF, '04X') # Mask
→to 16 bits

                print(f"        och[{och_index:2d}]: {hex_val}")
                # print(f"    Cycle {cycle_count}: nij={nij},
→och[{och_start}:{och_end}] = {psum_result.tolist()}")

            cycle_count += 1

        print(f"    Completed processing {input_matrix.size(1)} time steps
→for och_tile {och_t}")

    return psum_memory

def print_first_cycles_detailed(psum_memory, num_cycles=10):
    """
    Print detailed information for the first several cycles
    """
    device = psum_memory.device
    print(f"\n=== First {num_cycles} Cycles Detailed Output ===")

    count = 0
    for kij in range(9):
        for nij in range(16):
            if count >= num_cycles:
                return

            print(f"\nCycle {count}: kij={kij}, nij={nij}")
            print(f"    Partial sums for all output channels:")
            for och in range(16):
                psum_val = psum_memory[och, nij, kij].item()
                print(f"        och[{och}]: {psum_val}")

            count += 1

```

```

def accumulate_and_verify(psum_memory, weight_int, x_pad_in):
    """
    Accumulate partial sums and verify with PyTorch convolution
    """
    device = psum_memory.device
    weight_int = weight_int.to(device)
    x_pad_in = x_pad_in.to(device)

    print("\n=== Accumulating Partial Sums ===")

    # Accumulate across kernel elements (kij dimension)
    accumulated_psum = torch.sum(psum_memory, dim=2) # [16, 16]
    print(f"Accumulated psum shape: {accumulated_psum.shape}")

    # Reshape to spatial dimensions [16, 4, 4]
    final_output_simulated = accumulated_psum.view(16, 4, 4)
    print(f"Final output shape: {final_output_simulated.shape}")

    # Generate expected output using PyTorch
    print("\n=== Generating Expected Output ===")
    conv_ref = torch.nn.Conv2d(
        in_channels=16,
        out_channels=16,
        kernel_size=3,
        padding=0, # No padding in actual convolution
        bias=False
    ).to(device)

    # Set weights
    conv_ref.weight = torch.nn.Parameter(weight_int)

    # Perform convolution
    with torch.no_grad():
        expected_output = conv_ref(x_pad_in.unsqueeze(0)).squeeze(0) # Remove
        ↪ batch dimension

    print(f"Expected output shape: {expected_output.shape}")

    # Compare results
    diff = torch.abs(final_output_simulated - expected_output)
    max_diff = torch.max(diff)

    print(f"\n=== Verification Results ===")
    print(f"Max difference: {max_diff.item()}")
    print(f"Outputs match: {torch.allclose(final_output_simulated,
        ↪ expected_output, atol=1e-5)}")

```

```

    return final_output_simulated, expected_output

# Example usage with your data:
def main_simulation():
    """
    Main function to run the complete simulation
    """
    print("Starting systolic array simulation...")

    # Assuming you have these variables from your notebook:
    # x_pad_in: [16, 6, 6] - input activations
    # weight_int: [16, 16, 3, 3] - quantized weights

    # Make sure both tensors are on CUDA
    global x_pad_in, weight_int
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Move tensors to device if they exist
    if 'x_pad_in' in globals():
        x_pad_in = x_pad_in.to(device)
    else:
        print("Creating sample x_pad_in data...")
        x_pad_in = torch.randint(0, 4, (16, 6, 6), device=device).float() # ↵
        ↪ 2-bit activations (0-3)

    if 'weight_int' in globals():
        weight_int = weight_int.to(device)
    else:
        print("Creating sample weight_int data...")
        weight_int = torch.randint(-8, 8, (16, 16, 3, 3), device=device).
        ↪ float() # 4-bit weights

    print(f"Using device: {device}")

    # Run detailed simulation
    psum_memory = simulate_systolic_array_detailed(x_pad_in, weight_int, ↵
    ↪ och_tile_size=8)

    # Print first cycles in detail
    print_first_cycles_detailed(psum_memory, num_cycles=5)

    # Accumulate and verify
    final_sim, expected = accumulate_and_verify(psum_memory, weight_int, ↵
    ↪ x_pad_in)

    return psum_memory, final_sim, expected

```



```

# Alternative: If you want to run with your specific data from the notebook
def run_with_notebook_data():
    """
    Run simulation with data from your notebook
    """
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    try:
        print("Using data from notebook context...")
        # Make sure these variables are available and on the correct device
        global x_pad_in, weight_int, x_pad

        # Ensure all tensors are on the same device
        if 'x_pad_in' not in globals():
            # Create x_pad_in from x_pad if needed
            x_pad_in = x_pad[0].to(device) # Taking first batch item

        x_pad_in = x_pad_in.to(device)
        weight_int = weight_int.to(device)

        print(f"Tensors moved to device: {device}")
        print(f"x_pad_in device: {x_pad_in.device}")
        print(f"weight_int device: {weight_int.device}")

        psum_memory = simulate_systolic_array_detailed(x_pad_in, weight_int)
        print_first_cycles_detailed(psum_memory, num_cycles=8)
        final_sim, expected = accumulate_and_verify(psum_memory, weight_int,
↪x_pad_in)
        return psum_memory, final_sim, expected
    except NameError as e:
        print(f"Notebook data not fully available: {e}")
        print("Running with sample data instead...")
        return main_simulation()
    except Exception as e:
        print(f"Error during simulation: {e}")
        print("Running with sample data instead...")
        return main_simulation()

# To execute the simulation:
# Uncomment one of the following lines:

# For using your notebook data:
psum_memory, final_output, expected_output = run_with_notebook_data()

# For using sample data:
# psum_memory, final_output, expected_output = main_simulation()

```

```

# To examine specific parts of the result:
# print("Partial sums for first kernel element:")
# print(psum_memory[:, :, 0]) # All output channels, all positions, kij=0

# Additional debugging function to check tensor devices
def check_tensor_devices():
    """
    Check and print the devices of all relevant tensors
    """
    tensors_to_check = ['x_pad_in', 'weight_int', 'x_pad', 'W', 'X']

    print("=== Tensor Device Check ===")
    for tensor_name in tensors_to_check:
        if tensor_name in globals():
            tensor = globals()[tensor_name]
            if hasattr(tensor, 'device'):
                print(f"{tensor_name}: {tensor.device}")
            else:
                print(f"{tensor_name}: Not a tensor or no device attribute")
        else:
            print(f"{tensor_name}: Not found in global scope")

# Call this to check tensor devices:
# check_tensor_devices()

```